Linux Contents

Module 1: Linux Basics
Topics:
- **What is Linux?**
- **Distributions (Ubuntu, CentOS, RHEL)**
- **Terminal basics**
- **File system structure (/, /home, /var, /etc)**

Module 2: File Permissions & Users
Topics:
- **Users & groups**
- **File permissions (rwx)**
- **Ownership**

Module 3: Process & System Management
Topics:
- **Process lifecycle**
- **Foreground / Background**
- **System monitoring**

Module 4: Disk & Storage
Topics:
- **Disk structure**
- **Mounting**
- **LVM basics**

Module 5: Networking
Topics:
- **IP, DNS, Gateway**
- **Ports & protocols**

Module 6: Services & Logs
Topics:
- **systemd**
- **Logs**

Module 7: Security Basics
Topics:
- **SSH**
- **Firewall**
- **SELinux basics**

Module 8: Bash Scripting
Topics:
- **Variables**
- **Loops**
- **Conditions**

Linux Filesystem

1. What is FileSystem?

A file system is a method used by the operating system to organize, store, and manage data on a storage device like a hard disk. It defines how data is stored, accessed, and retrieved

2. Windows vs Linux

Windows path: C:\Users\<user_name>\Downloads

Linux: /home/<user_name>/<file_name>

Windows is case insensitive and Linux is case sensitive

3. Types of File Systems (Common)

ext4 → most common in Linux
xfs → used in enterprise systems
ntfs → Windows
fat32 → USB drives

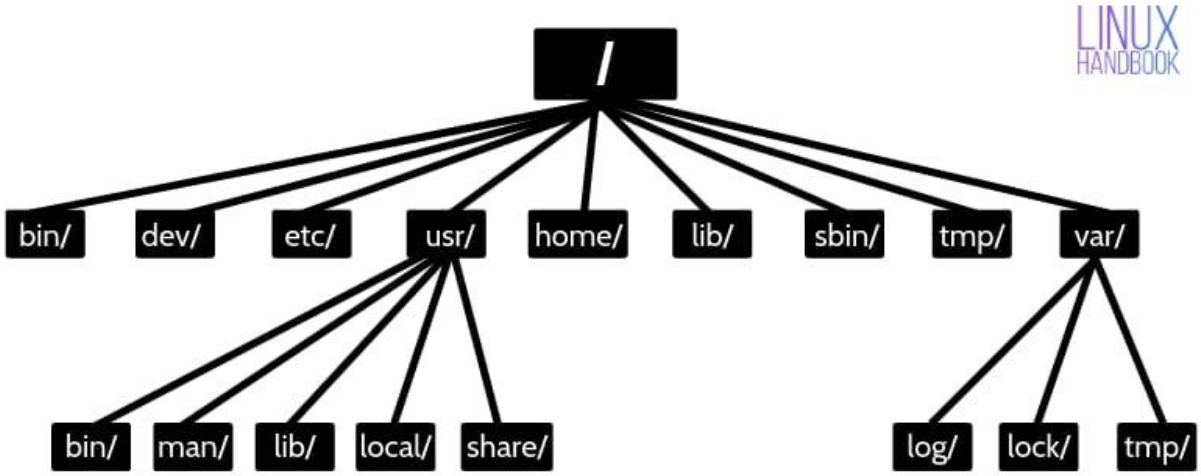
4. Difference between ext4 and XFS

ext4 and XFS are both Linux file systems. ext4 is more general-purpose and widely used, while XFS is designed for high performance and large-scale systems, especially for handling large files and parallel operations.

Key Differences

Feature	ext4	xfs
Type	General purpose	High-performance
Performance	Good	Better for large data
File Size Support	Up to ~16TB	Very large (petabytes)
Resize	Can shrink & grow	Only grow (cannot shrink)
Speed	Moderate	Faster for large files
Journaling	Yes	Yes (more efficient)
Use Case	Default Linux FS	Enterprise / Big data

5. Linux File System Structure



/

Root directory — the top of the filesystem; everything starts from here.

/bin

Essential user commands (like ls, cp, mv) required for basic system operation.

/sbin

System/admin commands used for system maintenance (mainly for root user).

/etc

Stores system-wide configuration files (users, services, network configs).

/home

Contains personal directories for all regular users.

/root

Home directory of the root (admin) user.

/var

Stores variable data like logs, cache, mail, and application data.

/tmp

Temporary files; usually cleared automatically after reboot.

/usr

Contains user-installed applications, libraries, and documentation.

/opt

Used for installing third-party or optional software.

/dev

Contains device files representing hardware (disks, USB, etc.).

/proc

Virtual filesystem showing process and system information (CPU, memory, running processes).

/boot

Contains bootloader files and Linux kernel required for system startup.

/mnt

Temporary mount point for manually mounted filesystems.

/media

Auto-mounted removable media like USB drives and CDs.

/lib & /lib64

Shared libraries required by system programs and binaries.

Users & Groups

1. What is a User?

A **user** is a person or system account that can log into Linux and perform actions.

Example:	**Login Command**
• Root -> admin user	sudo su
• Steve -> normal user	su - <user_name>

2. What is a Group?

A **group** is a collection of users.

Uses:

- To manage permission easily.
- Instead of giving access to each user, you give access to a group.

Example:

- Group: Developers
- Users: ram, shyam , alex

Commands:

```
useradd john      # create user
adduser john      # advanced mode
groupadd dev      # create group
userdel john      #delete user
groupdel dev      #delete group
usermod -aG dev john   # add user to group
gpasswd –d john dev  #remove john from dev group
```

id <user_name>

output: uid=1001(user_name) gid=1004(group_name) groups=1004(group_name),1001(dev)

User ID | **Group ID**

File Permissions

File Permissions (rwx)

Every file/directory has permissions that define: Who can read, write, or execute it

Three Types of Permissions:

Symbol	Meaning	Example
r	Read	View file
w	Write	Edit file
x	Execute	Run file

Three Levels of Access:

Level	Meaning
User (u)	Owner of file
Group (g)	Group members
Others (o)	Everyone else

Each permission has a number:

Permission	Value
r (read)	4
w (write)	2
x (execute)	1

Example1:

-rwxr-xr--

Section	**Meaning**
Rwx	Owner → full access
r-x	Group → read & execute
r--	Others → read only

Easy Trick: Always read in groups of 3:

rwx | r-x | r--

u g o

<u>Change Persmission:</u>

chmod 755 file.txt

7 → rwx

5 → r-x

5 → r-x

<u>Example2:</u>

mkdir /tmp/shared

chmod 1777 /tmp/shared

1 777

| └── Normal permissions (rwx for all)

└─── Special permission (sticky bit)

Owner → full access

Group → full access

Others → full access

So everyone can read, write, create files

What is that 1? (Sticky Bit)

1 = Sticky Bit

What Sticky Bit does

Rule:

Users can create files, but can only delete their own files

<u>Example3:</u>

chmod g+s /tmp/project

This command sets the SGID (Set Group ID) on the /tmp/project directory.

What SGID does?

When SGID is applied on a directory:

All new files and folders created inside will inherit the same group as the directory.

Without SGID

/project → group = dev

User john creates file:

file.txt → group = john

Problem:

Group mismatch

Team cannot access properly

With SGID (After chmod g+s)

/project → group = dev

User john creates file:

file.txt → group = dev

Solution:

All team members (dev group) can access

Ownership

Every file has:

- Owner (user)
- Group

Example1:

-rw-r--r-- 1 john dev file.txt

- Owner = john
- Group = dev

Change Ownership

- chown john file.txt **# change owner**
- chown john:dev file.txt **# change owner + group**

Example2:

-rwxr----- 1 john dev script.sh

- Owner (john) → full access
- Group (dev) → read only
- Others → no access

Example3:

- chown john:dev file.txt
- chmod 770 file.txt
- Owner + group → full access
- Others → no access

Users = who is using system

Groups = collection of users

Permissions = what they can do

Ownership = who controls the file

#Recursive flag: It inherits the command property.

Ex: file path: /home/ubuntu/file.txt

chmod 755 /home/ubuntu #it will change permission for only ubuntu dir

chmod -R 755 /home/ubuntu #it will change permission for ubuntu along with all files & dirs inside ubuntu.

-R → recursive (safe meaning across commands, applicable everywher)

-r → depends on command (cannot use in chmod, chown)

Process Lifecycle

How a process lives & dies

1. What is a Process?

A **process** is equal to a running program

Example:
sleep 100 #command
This creates a process

2. Lifecycle stages

A process goes through different states:

New → Ready → Running → Waiting → Terminated

a. New (Created)

Process is created

Example: you type a command

b. Ready

Waiting for CPU

In queue

c. Running

CPU is executing it

d. Waiting (Sleeping)

Waiting for I/O (disk, network, etc.)

Example:

sleep 10 #command

It is in **waiting state**

e. Terminated

Process finished or killed

How to check process states

ps aux #command

Look at column:

- R → Running
- S→ Sleeping
- Z → Zombie (important)

3. What is Zombie process?

-> Process is dead but still in table

-> Parent didn’t read its status

Kill a process

kill <PID> #command

kill -9 <PID> # force kill command

Foreground vs Background Process

1. Foreground process

-> Runs in terminal and blocks it

Example:

sleep 20 #command

You cannot type anything until it finishes

2. Background process

-> Runs in background → terminal is free

Example:

sleep 100 & #command

Output:

[1] 1234

-> 1 → job id

-> 1234 → PID

Move running process to background

a. Run:

sleep 100 #command

b. Press:

Ctrl + Z

Suspends process

c. Then:

bg #command

Runs in background

Bring back to foreground

fg #command

See background jobs

jobs #command

Summary

Type	Behavior
Foreground	Blocks terminal
Background	Runs freely

System Monitoring (Very Important)

1. Why monitoring?

To check:

-> CPU usage

-> Memory usage

-> Running processes

-> System load

a. **Check CPU usage**

i. top #command

Shows:

-> CPU %
-> Memory
-> Running processes

ii. Better version

htop #command

More user-friendly (install if not present)

b. **Check memory**

free -h #command

Output:

-> total

-> used

-> free

c. **Check processes**

ps aux #command

d. Check specific process

ps aux | grep nginx #command

e. System load

uptime #command

Shows load average

f. Disk usage

df –h #command

g. Real-time process tracking

top –p <PID> #command

htop Command explanation:

1. CPU Colors Explanation

Color	Meaning	Good/Bad
Green	User processes	✅ Normal
Red	Kernel/system	⚠️ Watch
Blue	Nice processes	✅ Fine
Yellow	I/O wait	❌ Disk issue
Purple	Steal time	❌ VM issue
Grey	Idle	✅ Good

Case 1: Mostly Green

-> Normal application load
✓ Healthy system

Case 2: High Red

Kernel busy
Possible causes:

-> Heavy I/O
-> Network issues
-> Driver problem

Case 3: High Yellow

Disk is slow
Common in:
-> Database servers
-> Logging-heavy systems

Case 4: High Purple

Cloud issue

-> Shared CPU overloaded

2. Memory Colors Explained

Green → Used Memory (Applications)
Memory used by:
-> Running programs (Java, Python, etc.)
-> Services (nginx, mysql)
This **is real usage**

Blue → Buffers
Memory used by kernel for:
-> Temporary data
-> Disk operations
✓ Small portion usually
✓ Helps system performance

Yellow → Cache

Memory used for:

-> File caching

-> Frequently accessed data

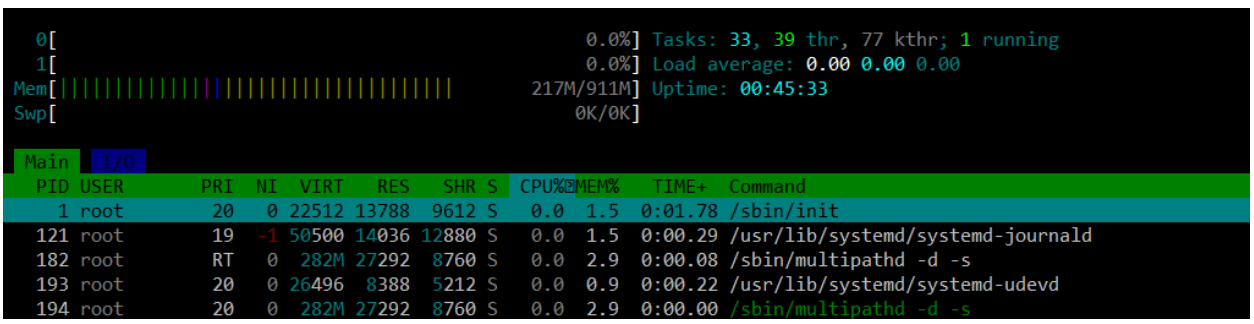
✓ VERY IMPORTANT:

This memory can be freed anytime if needed

Grey → **Free Memory**

-> Completely unused RAM

Color	Meaning	Important?
Green	App memory	✅ Yes
Blue	Buffers	Minor
Yellow	Cache	✅ Very important
Grey	Free	Not critical



a. PID: Process ID

b. User: Name of users

c. PRI(Priority): Lower number -> Higher priority, kernel decides this based on NI

d. NI (Nice value): controls priority manually. Ex: -20(highest priority) , 19(Lowest Priority), 0 (Default Priority)

e. VIRT(Virtual Memory): Total memory used by process including everything (RAM, SWAP, Shared lib, reserved memory). *Note: VIRT is often large and misleading.*

f. RES (Resident memory): Actual RAM used right now.

g. SHR (Shared Memory): Memory shared with other processes. Ex: shared libraries, common resources.

h. S (State): Process state.

top command explanation:

```
top - 11:48:58 up  1:02,  3 users,  load average: 0.00, 0.00, 0.00
Tasks: 116 total,   1 running, 115 sleeping,   0 stopped,   0 zombie
%Cpu(s):  0.2 us,  0.3 sy,  0.0 ni, 99.5 id,  0.0 wa,  0.0 hi,  0.0 si,  0.0 st
MiB Mem :    911.5 total,    271.6 free,    382.3 used,    416.1 buff/cache
MiB Swap:      0.0 total,      0.0 free,      0.0 used.    529.2 avail Mem
```

i) **11:48:58** -> current system time

ii) **up 1:02min** -> system running since last reboot

iii) **3 users** -> 2 logged-in users

iv) **load average** -> system load.

0.00, 0.00, 0.00 -> Last 1 min, 5 min, 15 min resp.

Ex: 0.00 -> no load (idle system), 1.00 -> CPU fully used, > CPU cores -> overloaded.

v) **Tasks:**

```
Tasks: 116 total,   1 running, 115 sleeping,   0 stopped,   0 zombie
```

Field	Meaning
total	total processes
running	currently using CPU
sleeping	waiting (normal)
stopped	paused (Ctrl+Z)
zombie	dead but not cleaned

Currently 1 running -> very light load.

115 sleeping -> normal

0 zombie -> healthy system

vi) **CPU usage:**

```
%Cpu(s):  0.2 us,  0.3 sy,  0.0 ni, 99.5 id,  0.0 wa,  0.0 hi,  0.0 si,  0.0 st
```

current -> 99.5 id CPU is 99.5% idlle

Field	Meaning
us	User processes
sy	System (kernel)
ni	Nice (low priority)
id	Idle
wa	Waiting for I/O
hi	Hardware interrupts
si	Software interrupts
st	Stolen (VM usage)

vii) **Memory (RAM):**

```
MiB Mem :    911.5 total,    271.6 free,    382.3 used,    416.1 buff/cache
```

buff/cache is not wasted. Linux is using free RAM as cache to improve performance.

Field	Meaning
total	total RAM
free	unused RAM
used	actively used
buff/cache	cached memory

viii) **Swap:**

```
MiB Swap:      0.0 total,      0.0 free,      0.0 used.    529.2 avail Mem
```

disk used as RAM (backup memory.)
current -> No swap configured, available memory for swap: 592.2 MB

Inode, Kernel, and Daemon

Inodes

Inode is a data structure in Linux that stores metadata about a file such as permissions, owner, size, and disk location. It does not store the file name or actual file content.

Key Points:
- Each file has a unique inode number
- Stores metadata like size, permissions, owner
- Does not store file name or content
- Used by OS to locate file data

Inode Number

An inode number is a unique identifier assigned to every file in a Linux filesystem.
It is used by the system to locate the file's metadata and actual data on disk.
Ex: 123456 file.txt -> inode_number file_name

Commands:
ls -i file.txt
df -i
stat file.txt

Important Concept:
Disk can be full due to space (df -h) or inode exhaustion (df -i)

Kernel

The kernel is the core component of the Linux OS that interacts directly with hardware and manages system resources like CPU, memory, and devices. It acts as a bridge between applications and hardware.

Key Functions:
- CPU management
- Memory management
- Device handling
- System calls execution

Working Flow:
Application → Kernel → Hardware

Example:
When a command like 'ls' is executed, the kernel interacts with the disk and returns the file list.

Daemon

A daemon is a background process that runs continuously to provide system services or perform tasks. It usually starts at boot and runs without user interaction.

Examples:
sshd → SSH service
cron → scheduled tasks
httpd/nginx → web server

Commands:
ps -ef | grep sshd
systemctl status ssh

Kernel vs Daemon

Kernel:
- Core of OS
- Manages hardware
- Runs in kernel space

Daemon:
- Background service
- Provides system/application services
- Runs in user space

Summary

Inodes store file metadata, kernel manages hardware/resources, and daemons provide background services.

Disk Structure

Definition

Disk structure refers to how a storage device is organized into partitions and file systems so the operating system can store and manage data efficiently. It defines how raw disk space is divided and used.

Basic Idea

A disk (like HDD/SSD) is not used directly. It is divided into partitions, and each partition is formatted with a file system.

Disk Structure Flow

Disk → Partition → File System → Mount → Access Files

Important Components

1. Disk
Physical storage device
Example: /dev/sda, /dev/sdb

2. Partition
Logical division of disk
Example: /dev/sda1, /dev/sdb1

3. File System
Organizes data
Example: ext4, xfs

4. Mount Point
Location where disk is attached
Example: /mnt, /data

Example

Suppose you have a new disk /dev/sdb
1. Create partition → /dev/sdb1
2. Format it → ext4 or xfs
3. Mount it → /data
4. Store files

Visual Understanding

```
/dev/sdb (Disk)
  └── /dev/sdb1 (Partition)
      └── ext4 (File System)
          └── \data (Mount Point)
                └──Files & Data
```

Practical Commands

Check disks:
lsblk

Create partition:
fdisk /dev/sdb

Format disk:
mkfs.ext4 /dev/sdb1

Mount disk:
mount /dev/sdb1 /data

Real-Life Analogy

Disk = Land
Partition = Plots
File system = Layout plan
Files = Houses

Key Understanding

- Disk must be partitioned before use
- Partition must be formatted
- Then it can store data

Summary

Disk structure is the organization of a disk into partitions and file systems so data can be stored and accessed properly.

Mounting

Definition

Mounting is the process of attaching a filesystem (disk or partition) to a specific directory so that it becomes accessible to the system. Without mounting, the OS cannot access the data on that disk.

Basic Idea

In Linux, storage is not used directly. It must be mounted to a directory (mount point) before accessing files.

Mounting Flow

Disk → Partition → File System → Mount → Access Files

Example

/dev/sdb1 is mounted to /data.
After mounting, /data shows the contents of /dev/sdb1.

Visual Structure

/dev/sdb1 (disk partition)
↓
mounted on
↓
/data (directory)

Temporary vs Permanent Mount

Temporary Mount:
mount /dev/sdb1 /data (lost after reboot)

Permanent Mount:
Edit /etc/fstab and add:
/dev/sdb1 /data ext4 defaults 0 0

Important Commands

Check mounted disks:
mount
lsblk

Unmounting

umount /data
or
umount /dev/sdb1

Important Points

Directory must exist before mounting.
Cannot unmount if the disk is in use.
Always verify after mounting.

Scenario

If disk is attached but not visible:
1. lsblk
2. mount
3. Check /etc/fstab

Summary

Mounting connects a disk or partition to a directory so the system can access its data. Without mounting, storage devices remain unusable.

Practical: AWS EBS Volume Setup Guide (Linux)

Step 1: Verify Attached Volume
Run: **lsblk**
You will see new disk like:
nvme1n1 10G disk

Step 2: Create Partition
Run: **sudo fdisk /dev/nvme1n1**

Inside fdisk:
n → new partition
p → primary
1 → partition number
Enter → default start
Enter → default end
w → save

Step 3: Verify Partition: **lsblk**
Output:
nvme1n1
└**nvme1n1p1**

Step 4: Format Filesystem: **sudo mkfs.ext4 /dev/nvme1n1p1**

Step 5: Create Mount Point: **sudo mkdir /data**

Step 6: Mount Volume: **sudo mount /dev/nvme1n1p1 /data**

Step 7: Verify Mount: **df -h**

Step 8: Make Mount Permanent
Edit **fstab:**
sudo nano /etc/fstab
Add line: **/dev/nvme1n1p1 /data ext4 defaults,nofail 0 2**

Test: **sudo mount -a**

Step 9: Store Data
cd /data
echo "Hello AWS Volume" > file.txt
<u>Explanation of fstab Entry</u>

Line: **/dev/nvme1n1p1 /data ext4 defaults,nofail 0 2**

Structure:
<device> <mount_point> <filesystem> <options> <dump> <fsck>

1. Device
/dev/nvme1n1p1
Disk partition
nvme1n1 = disk
p1 = partition 1

2. Mount Point
/data
Location where disk is mounted
Access using: cd /data

3. Filesystem
ext4
Type of filesystem
Common in Linux

4. Options
defaults,nofail
defaults includes:
rw (read/write), suid, dev, exec, auto, nouser, async

nofail: Prevents boot failure if disk is missing

5. Dump
0: Backup option (usually not used)

6. FSCK Order
2
Filesystem check order
1 = root (/)
2 = other filesystems

Practical2: Resize Disk

Step1: increase volume in AWS
10GB → 20GB

Step2: verify new size
lsblk
You'll see disk increased but partition same.

Step3: Extend partition
sudo growpart /dev/nvme1n1 1

Step4: resize filesystem
For ext4:
sudo resize2fs /dev/nvme1n1p1

Done
Check: **df –h**

Unmount command: sudo umount <mountpoint>

LVM (Logical Volume Manager) Basics

Definition

LVM (Logical Volume Manager) is a storage management system that allows flexible allocation and resizing of disk space. It helps combine multiple disks and manage them as a single logical storage.

Why LVM is Used

Normal partitions have fixed size and are hard to resize.
LVM allows easy resizing, combining multiple disks, and better storage management.

LVM Structure

Disk → PV → VG → LV → Filesystem → Mount

Components of LVM

PV (Physical Volume): A physical disk or partition used in LVM (e.g., /dev/sdb).

VG (Volume Group): A pool of storage created by combining multiple PVs (e.g., vg_data).

LV (Logical Volume): A virtual partition created from VG, used like a normal disk (e.g., lv_data).

Visual Structure

```
/dev/sdb + /dev/sdc → VG (vg_data)
                ↓
            LV (lv_data)
                ↓
              /data
```

Useful Commands

pvs → Check physical volumes
vgs → Check volume groups
lvs → Check logical volumes

Advantages

Flexible storage management
Easy resizing
Combine multiple disks
Suitable for production environments

Important Points

LVM sits between disk and filesystem.
Logical volumes behave like normal partitions.
Commonly used in servers and cloud environments.

Summary

LVM allows flexible disk management by combining physical storage into logical volumes that can be resized easily.

Practical Steps

1. Create Physical Volume: sudo pvcreate /dev/nvme1n1, **check:** sudo pvs

2. Create Volume Group: sudo vgcreate vg_data /dev/ nvme1n1, **check:** sudo vgs

3. Create Logical Volume: lvcreate -l 100%FREE -n lv_data vg_data, **check:** sudo lvs

4. Format: mkfs.ext4 /dev/vg_data/lv_data

5. Mount: mkdir /data -> mount /dev/vg_data/lv_data /data, **check:** df –h

lvcreate -l 100%FREE -n lv_data vg_data

-L: Fixed size, Ex: 2G, 500M

-l: Relative size, Ex: 100%FREE, 50%FREE

Extend Logical Volume

1. Resize Physical Volume (PV): sudo pvresize /dev/nvme1n1

2. Now Extend Logical Volume (LV): sudo lvextend -l +100%FREE /dev/vg_data/lv_data

3. Resize Filesystem: sudo resize2fs /dev/vg_data/lv_data **(FOR xfs:** sudo xfs_growfs /data**)**

IP Address

Definition

An IP address (Internet Protocol address) is a **unique numerical identifier** assigned to a device on a **network**. It is used to **identify** and **communicate** between devices over a network.

1. Why IP Address is Needed

To identify devices
To send and receive data
To enable communication between systems

2. Simple Understanding

Think:

IP address = house address
Internet = city
Without address → data cannot reach the correct device

3. Example

192.168.1.10 (This is a typical IP address)

4. Types of IP Address

1. IPv4 (Most Common)

An IPv4 address is made up of 32 bits, divided into 4 octets.
Each octet contains 8 bits and is separated by dots.

Octet: An octet is a group of 8 bits in an IP address.

Structure of IPv4

192.168.1.10

Breakdown:

192 .168 .1 .10

Octet1 Octet2 Octet3 Octet4

Range of Each Octet

Each octet can have values: 0 to 255

Because: 2^8 = 256 values (0–255)

2. IPv6

IPv6 (Internet Protocol version 6) is the newer version of IP addressing designed to replace IPv4.
It uses 128 bits, allowing a much larger number of unique IP addresses.

Structure of IPv6

IPv6 is made of 8 groups (blocks).
Each group = 16 bits

Example: 2001:0db8:85a3:0000:0000:8a2e:0370:7334

Breakdown:

```
2001 : 0db8 : 85a3 : 0000 : 0000 : 8a2e : 0370 : 7334
  |      |      |      |      |      |      |      |
 Block1 Block2 Block3 Block4 Block5 Block6 Block7 Block8
```

How It Is Written

Hexadecimal numbers (0–9, a–f)
Colon : separator

Address Capacity

IPv4: 32 bits → ~4.3 billion IPs
IPv6:128 bits → extremely large (almost unlimited)

5. Public vs Private IP

Private IP

Used inside local network
Example: 192.168.x.x, 10.x.x.x, 172.16.x.x

Public IP

Used on internet. Provided by ISP

6. Static vs Dynamic IP

Static IP

Fixed address. Does not change
Use: Servers

Dynamic IP

Changes automatically. Assigned by DHCP
Use: Client devices (Desktop, Laptop, Mobile etc.)

7. Check IP in Linux

ip a or **hostname -I**

8. Real Example

- When you open a website:

Your system sends request from your IP and Server responds to your IP

9. Important Points

Every device must have an IP
No two devices can have same IP in same network
Used in all networking

Final Summary

An IP address is a unique number used to identify devices and enable communication in a network.

IP Classes

IP classes are a way to divide IPv4 addresses into different categories based on size and usage of networks.
They help in organizing IP addresses for small, medium, and large networks.

1. Why IP Classes Exist

To manage IP address allocation
To support different network sizes
To separate network and host portions

2. Main IP Classes

There are 5 classes: i) Class A ii) Class B iii) Class C iv) Class D v) Class E

3. Class A

Range: 1.0.0.0 – 126.255.255.255
Default Subnet: 255.0.0.0
Use: Very large networks
Example: 10.0.0.1

4. Class B

Range: 128.0.0.0 – 191.255.255.255
Default Subnet: 255.255.0.0
Use: Medium-sized networks
Example: 172.16.0.1

5. Class C

Range: 192.0.0.0 – 223.255.255.255
Default Subnet: 255.255.255.0
Use: Small networks
Example: 192.168.1.1

6. Class D

Range: 224.0.0.0 – 239.255.255.255
Use: Multicast (group communication)

7. Class E

Range: 240.0.0.0 – 255.255.255.255

Use: Experimental / research

8. Quick Trick to Identify Class

Look at **first octet**

First Number	Class
1–126	A
128–191	B
192–223	C
224–239	D
240–255	E

9. Easy Memory Trick

A → Big network

B → Medium

C → Small

Final Summary

IP classes divide IPv4 addresses into groups (A, B, C, D, E) based on network size and purpose.

What about 127.x.x.x ?

Definition

The IP range 127.0.0.0 – 127.255.255.255 is reserved for loopback addresses.
It is used by a system to communicate with itself and is not used for normal networking.

1. Why 127 is Missing in Class A

Earlier we said **Class A: 1 – 126.** Because **127 is reserved.** So, it is excluded from usable **Class A range**

2. Most Common Loopback IP

127.0.0.1. It is Also called **localhost**

3. What is Loopback?

It means Your system talks to **itself**

Example: ping 127.0.0.1

Result:

Tests local network stack. Does NOT go to internet

4.Usage

Testing network services
Checking if TCP/IP is working
Local development (apps running on same machine)

Final Summary

127.x.x.x is reserved for loopback (localhost) and is not used as a normal Class A network range.

IP Address Count in Each Class

Formula

Total IPs = 2^(number of host bits)

Example:

Class A

Default subnet: 255.0.0.0

Network bits: 8

Host bits: 24

Total IPs: 2^24 = 16,777,216

Usable IPs: 16,777,214 (Subtract network IP + broadcast IP)

Note: Class D *has no usable* ***host IP*** *concept. It is only used for multicast. And* ***Class E*** *is only reserved for research.*

Network Address

A network address is the first IP address in a network, used to identify the entire network.
It cannot be assigned to any device.

Broadcast Address

A broadcast address is a special IP address used to send data to all devices in a network at once.
It is the last IP address of a subnet and cannot be assigned to any device.

Subnet

A subnet is a smaller division of a network created by splitting a larger network into multiple smaller networks.
It helps in better management, security, and efficient use of IP addresses.

Why Subnetting is Needed

- Reduce network congestion
- Improve performance
- Increase security
- Better IP management

Example: 192.168.1.0/24 | **Total IPs:** 256

Subnet Mask

Subnet mask defines which part of IP is network and which is host.

Example: 255.255.255.0

CIDR Notation

CIDR (Classless Inter-Domain Routing) is a shorthand way to represent the subnet mask using a number.

Example: /24.

VLSM (Variable Length Subnet Mask)

VLSM is a subnetting technique of creating different sizes of subnets.

Why VLSM is Needed

Avoid IP wastage
Different departments need different IP sizes
More efficient than fixed subnetting

Example: 192.168.1.0/24, 192.168.1.0/26, 192.168.1.0/27

Subnet count Formula:

(2^[Total_Octet_Size] - [CIDR Notation])

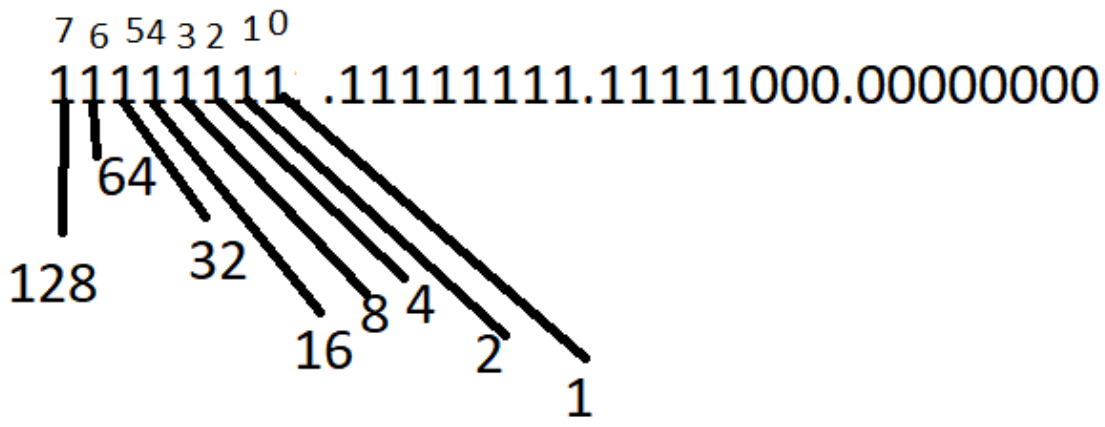
Example:
How many IPs in this subnet range 192.168.1.0/21?

Step1: 32 – 26 = 6
Step2: 2^6 = 64

Number of IPs: 64
Number of usable IP: 64 – 2 = 62

Subnet mask count Formula:

128+64+32+16+8+4+2+1 = 255 – size of 1 octet



Example:
255-7 = 248| 255.255.248.0

DNS(Domain Name System)

Definition

DNS (Domain Name System) is a system that converts human-readable domain names into IP addresses. It allows users to access websites using names instead of remembering IP addresses

1. Why DNS is Needed

Humans remember: google.com
Machines understand: **142.250.183.14**

DNS converts: Name → IP

2. Simple Understanding

DNS = Phonebook of Internet
Domain = Name
IP = Phone number

3. Types of DNS Servers

i) Recursive Server (DNS Resolver)

A DNS resolver is a server that receives a domain name request and finds the corresponding IP address by querying other DNS servers.
It acts as the middleman between the user and the DNS system.

Where Resolver Exists

ISP DNS (Airtel, Jio etc.)
Public DNS: 8.8.8.8(google), 1.1.1.1(Cloudflare)

Example:
Search: google.com
Resolver = helper who finds the IP for you

ii). Root Server (Global root servers)

Global root servers are the top-level DNS servers that handle the first step of DNS resolution and direct queries to the correct TLD servers.

There are 13 logical root server systems, distributed worldwide for reliability and performance.

a). Why Only 13 Root Servers?

- Historically, DNS used limited packet size (512 bytes). So, Only 13 root server addresses could fit
- Important, These are 13 logical servers, not just 13 machines.

b). Names of Root Servers

They are labeled:
A.root-servers.net
B.root-servers.net
C.root-servers.net
...
M.root-servers.net
Total: A to M = 13 root servers

c). Who Manages Them?

Different organizations manage them, for example: ICANN, Verisign, NASA

iii). TLD Server

A TLD (Top-Level Domain) server is a DNS server that manages domain extensions like .com, .org, .in and directs queries to the correct authoritative DNS server.
It does not provide the final IP address, but points to where the domain is hosted.
Example: .com, .org, .in

iV). Authoritative DNS

An authoritative DNS server is the DNS server that holds the actual DNS records of a domain and provides the final answer (IP address).
It is the source of truth for a domain..

Nameserver

A nameserver is a DNS server that responds to domain name queries and helps resolve domain names into IP addresses.
It stores or retrieves DNS records for a domain.

Example: ns1.example.com, ns2.example.com

Types of Nameservers

1. Recursive Nameserver (Resolver)
Example: 8.8.8.8
Does lookup for you

2. Authoritative Nameserver
Example: ns1.google.com

Stores real DNS records

4. How DNS Works (Step-by-Step)

Example:
You type: www.google.com

Step 1: Check Local Cache
System checks:
- Browser cache
- OS cache

Step 2: Query DNS Server
If not found: Request goes to DNS resolver

Step 3: DNS Resolution Process
Root Server → TLD Server → Authoritative Server

Step 4: Get IP
www.google.com → 142.250.x.x

Step 5: Connect to Server
Browser connects using IP

5. DNS Records (Definition)

DNS records are entries in a DNS database that map domain names to IP addresses or define how a domain behaves.
They tell the DNS server how to respond to queries.

i). A Record (Address Record)

An A (Address) record is a DNS record that maps a domain name to an IPv4 address.
It tells the system which IP address a domain should connect to.
Example: example.com → 192.168.1.10
Most commonly used

ii). AAAA Record

An AAAA record is a DNS record that maps a domain name to an IPv6 address.
It is similar to an A record, but it works for IPv6 instead of IPv4.
Example: example.com → 2001:db8::1

iii). CNAME Record (Canonical Name)

A CNAME (Canonical Name) record maps one domain name to another domain name.
It is used to create an alias instead of pointing directly to an IP address.
Example: www.example.com → example.com
Used for Aliases

iv). MX Record (Mail Exchange)

An MX (Mail Exchange) record is a DNS record that specifies which mail server is responsible for receiving emails for a domain.
It directs email traffic to the correct mail server.
Example: example.com → mail.example.com
Used in email systems

v). NS Record (Name Server)

An NS (Name Server) record specifies which DNS servers are authoritative for a domain.
It tells the internet where to go to find the DNS records of that domain.
Example: example.com → ns1.example.com

vi). PTR Record (Pointer Record)

Maps IP address back to domain name (reverse DNS).
Example: 192.168.1.10 → example.com

vii). TXT Record

Stores text information for domain verification and security.
Example: "v=spf1 include:_spf.google.com ~all"
Used for SPF, DKIM

viii). SOA Record (Start of Authority)

Contains administrative information about the domain.
It Includes Primary nameserver, Email, Serial number

6. Check DNS in Linux

nslookup google.com or dig google.com

7. DNS in System

File: /etc/resolv.conf

Example:
nameserver 8.8.8.8

8. Real Example

ping google.com

DNS resolves → IP → ping works

#Key Understanding

- DNS = Name → IP
- Works in background
- Essential for internet

Final Summary

DNS converts domain names into IP addresses so users can access websites easily.

Gateway

Definition

A gateway is the entry/exit point in a network that allows communication with other networks.
It is usually represented by an IP address configured on a device.

1. Simple Understanding

Your system → local network

Gateway → door to outside world

Without gateway, you can talk inside network. But cannot reach internet

Example

Your system IP: 192.168.1.10

Gateway: 192.168.1.1. This IP assigned in router only.

2. Why Gateway is Needed

- Connect different networks

- Send traffic outside local network

- Route packets to internet

3. How Gateway Works

Step-by-Step

1. You send request to external IP

2. System checks: “Is this in my network?”

Case 1: Same Network

192.168.1.20 → Direct communication

Case 2: Different Network

Request Send to gateway, then goes to server

Visual Flow

PC (192.168.1.10)
↓
Gateway (192.168.1.1)
↓
Internet
↓
Server

4. Routing Logic

Destination inside → send directly
Destination outside → send to gateway

5. Gateway Types

Gateways can be categorized based on their function and the type of traffic or networks they connect. Different gateways serve different purposes in a network.

i. Default Gateway

The main gateway used to send traffic outside the local network when no specific route exists.

What is a Specific Route?

A specific route is a routing rule that tells the system exactly how to reach a particular network using a specific gateway.

Router

A router is a network device that forwards packets between different networks based on routing tables.

Routing Table

A routing table is a list of rules used by a system or router to decide where to send network packets.
It maps destination networks to the next hop (gateway) and interface.

Example
Your system IP: 192.168.1.10

Routing table:
10.0.0.0/8 via 192.168.1.2 (If request destination is 10.0.0.0/8 then 192.168.1.2 gateway will use.)
default via 192.168.1.1

 # Check in Linux: ip route

Example:

default via 192.168.1.1 dev eth0

192.168.1.1 = gateway

ii. Network Gateway

A device that connects two different networks and routes traffic between them.

Example: Home router connecting LAN → Internet

iii. Application Gateway (Proxy Gateway)

A gateway that operates at application level and manages traffic for specific applications.

Example: Web proxy server can Filter traffic Cache content.

iv. NAT Gateway

A gateway that translates private IP addresses into public IP addresses for internet communication.

Example: Cloud NAT (AWS/Azure)**,** Home router NAT

v. VPN Gateway

A gateway that allows secure communication between networks over the internet using encryption.

Example: Site-to-site VPN**,** Remote access VPN

vi. Internet Gateway (Cloud Concept)

A gateway used in cloud environments to allow resources in a private network to access the internet.

Example: AWS Internet Gateway

7. Important Points

- Gateway must be in same network

- Usually router IP

- Only one default gateway

8. Common Issue

If gateway is wrong: Internet not working but local network works

Final Summary
A gateway is a router that connects your local network to other networks and acts as the exit point for external communication.

Port

Definition

A port is a logical communication endpoint used by a computer to identify a specific process or service on a system.
It allows multiple services to run on a single IP address.

1. Why Ports Are Needed

One system runs multiple services: Web server**,** SSH**,** Database

Ports help identify: Which service should receive the data

2. Port Number Range

Range	Type
0–1023	**Well-known ports**
1024–49151	**Registered ports**
49152–65535	**Dynamic/ephemeral**

3. Common Ports (Important)

Port	Service
22	SSH
80	HTTP
443	HTTPS
21	FTP
25	SMTP
53	DNS

4. Example

When you open website: google.com → port 443

Flow:

IP + Port = Destination
142.x.x.x:443

5. Check Open Ports (Linux)

netstat –tulnp or ss –tulnp

Key Difference
ss is faster, more efficient, and more detailed than netstat.

Key Points

- Ports identify services
- Same IP can have many ports
- Required for communication

Final Summary

Ports are logical endpoints that allow multiple services to run on a single system by identifying specific applications using port numbers.

Protocol

Definition

A protocol is a set of rules that define how data is transmitted and received over a network.
It ensures that devices can communicate correctly and understand each other.

Types of Protocols

1 Application Layer Protocols

Used by users/apps

Examples

Protocol	Purpose
HTTP	Web browsing
HTTPS	Secure web
FTP	File transfer
SMTP	Sending emails
DNS	Name → IP

2 Transport Layer Protocols

Control how data is delivered

◆ **TCP (Transmission Control Protocol)**

- Reliable, connection-based communication

Features
- Data guaranteed
- Ordered delivery
- Slower

◆ **UDP (User Datagram Protocol)**

Fast, connectionless communication

Features
- No guarantee
- Faster
- Used in streaming

3 Network Layer Protocols

Handle addressing and routing

Examples

Protocol	**Purpose**
IP	Addressing
ICMP	Error messages
ARP	IP → MAC

Protocol Stack Example

When you open a website:

Browser (HTTP)
↓
TCP (port 443)
↓
IP (destination address)
↓
Internet

Key Characteristics

- Defines rules
- Ensures communication
- Works in layers

Final Summary

Protocols are standardized rules that allow devices to communicate over a network, handling everything from data formatting to delivery.

ICMP

ICMP (Internet Control Message Protocol) is a network layer protocol used for sending error messages and operational information between devices.
It helps in diagnosing network issues and checking connectivity.

Simple Understanding

ICMP = network health checker

It tells:

Is destination reachable?

Is there an error?

Most Common Example

ping command

ping google.com

Uses ICMP

Sends: Echo Request

Receives: Echo Reply

ARP

ARP (Address Resolution Protocol) is used to find the MAC address of a device when its IP address is known.
It works within a local network to enable communication.

Simple Understanding

IP = logical address

MAC = physical address

ARP = "IP → MAC converter"

Why ARP is Needed

When your system sends data:

It needs MAC address, not just IP

Problem: “I know IP, but how to find MAC?”

Solution: ARP

MAC Address

A MAC (Media Access Control) address is a unique hardware identifier assigned to a network interface card (NIC).
It is used to identify devices within a local network.

IP address → logical (can change)

MAC address → physical (fixed)

Example of MAC Address

00:1A:2B:3C:4D:5E

Format:

6 pairs (hexadecimal)

48 bits total

Structure

```
00:1A:2B : 3C:4D:5E
  ↑          ↑
Vendor   Device ID
```

First part → manufacturer
Second part → unique device

Where MAC Address is Used

- Local network communication
- Used by switches
- Works at Data Link Layer (Layer 2)

How It Works

1. You send data to another device

2. System uses:

IP → MAC (via ARP)

3. Packet delivered using MAC address

Check MAC Address (Linux): ip a or ifconfig

Important Points

- Unique for each device
- Cannot be used on internet
- Used only in local network

MAC can be **Changed (spoofed).** But original is hardware-based

Final Summary

A MAC address is a unique hardware identifier used to identify devices in a local network and enable communication at the data link layer.

NIC

NIC (Network Interface Card) is a hardware component that allows a computer to connect to a network. It provides the interface between the device and the network (wired or wireless).

NIC = network adapter of your computer

Without NIC: No internet / no network connection

Examples

- Ethernet card (LAN cable)
- Wi-Fi adapter
- Virtual NIC (in VM/cloud)

Types of NIC

1 Wired NIC

Uses Ethernet cable

Example: eth0, ens5

2 Wireless NIC

Uses Wi-Fi

Example: wlan0

3 Virtual NIC

Used in VMs / cloud

Example:

AWS EC2 network interface

What NIC Contains

- MAC address (unique ID)
- Hardware for sending/receiving data

How NIC Works

1. Receives data from network
2. Converts signals → data
3. Sends to OS

(Reverse for sending)

Check NIC in Linux

ip a

Shows:
- NIC
- MAC address
- IP address

Key Points

Required for network connection

Works at Layer 1 & 2

Has MAC address

Final Summary

NIC is a hardware or virtual component that connects a device to a network and enables communication using a MAC address.

Firewall

A firewall is a security system that monitors and controls incoming and outgoing network traffic based on predefined rules.
It acts as a barrier between a trusted network and untrusted networks (like the internet).

Simple Understanding

Firewall = security guard at the gate

Allows safe traffic

Blocks harmful traffic

Example
Your server:

Port 22 (SSH) → allowed
Port 3306 (MySQL) → blocked
Only permitted traffic can enter

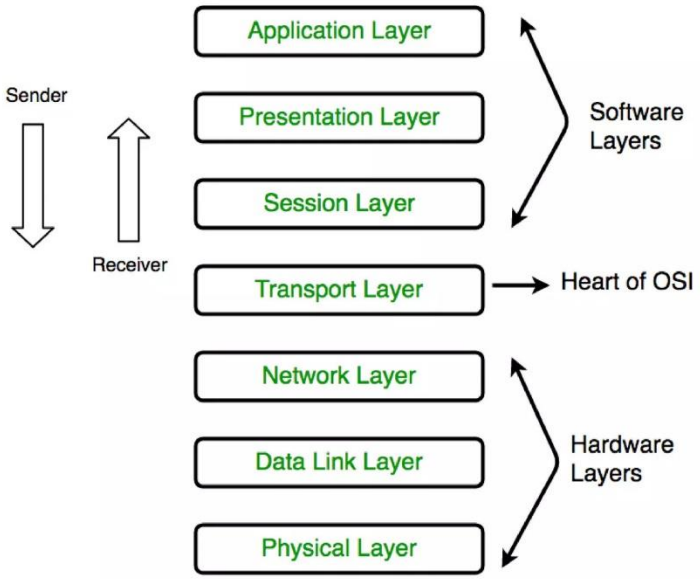
OSI Model

Definition

The OSI (Open Systems Interconnection) model is a 7-layer framework that describes how data is transmitted from one device to another over a network.
Each layer performs a specific function and passes data to the next layer.

The OSI model divides network communication into 7 layers, each handling a specific task from user interaction to physical data transmission.

Each OSI layer helps you identify where a problem or attack is happening and how to troubleshoot it.



Layer 7: Application Layer

The application layer provides an interface between user applications and the network.
It enables services like web browsing, email, and file transfer.
Examples: HTTP, HTTPS, FTP, DNS
Security Issues: Web attacks (SQL injection, XSS), DNS spoofing, API abuse
Troubleshooting: Website not loading, API errors, DNS resolution failure
Tools: curl, nslookup, dig

Layer 6: Presentation Layer

The presentation layer is responsible for data formatting, encryption, decryption and compression.
It ensures that data sent from one system can be properly understood by another.
Examples: SSL/TLS encryption, encoding formats
Security Issues: SSL/TLS certificate issues, Weak encryption, Data exposure
Troubleshooting: HTTPS not working, Certificate expired, SSL handshake failure

Layer 5: Session Layer

The session layer manages communication sessions between devices.
It establishes, maintains, and terminates connections during data exchange.
Examples: Login sessions, API sessions
Security Issues: Session hijacking, Session timeout issues
Troubleshooting: Login drops frequently, Session not maintained

Layer 4: Transport Layer

The transport layer provides end-to-end data delivery and ensures reliability.
It uses protocols like TCP (reliable) and UDP (fast) and manages port numbers.
Examples: TCP, UDP
Security Issues: Port scanning, SYN flood (DDoS), Unauthorized open ports
Troubleshooting: Service not reachable, Port blocked, Connection timeout
Tools: netstat, ss, telnet

Layer 3: Network Layer

The network layer handles logical addressing and routing of packets.
It determines the best path for data to travel using IP addresses.
Examples: IP, ICMP

Security Issues: IP spoofing**,** Routing attacks**,** ICMP abuse
Troubleshooting: Cannot reach network**,** Routing issue**,** Packet loss
Tools: ping, traceroute

Layer 2: Data Link Layer

The data link layer manages communication within the same local network.
It uses MAC addresses and ensures error-free transmission of frames.
Examples: Ethernet, ARP
Security Issues: ARP spoofing**,** MAC flooding**,** VLAN hopping
Troubleshooting: Cannot reach local devices, ARP mismatch**,** Switch issues
Tools: arp, ip neigh

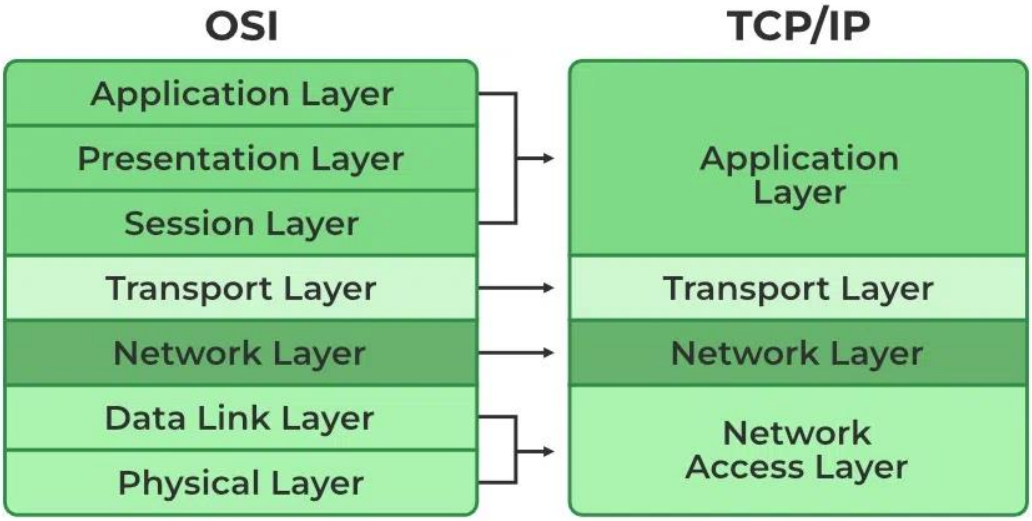
Layer 1: Physical Layer

The physical layer transmits raw bits over physical media.
It deals with cables, signals, and hardware components.
Examples: Ethernet cable, Wi-Fi signals
Security Issues: Cable tapping**,** Hardware damage
Troubleshooting: No network connection**,** Cable unplugged**,** Interface down
Check: Cable, NIC status, link light

TCP/IP Model

Definition

- TCP/IP (Transmission Control Protocol/Internet Protocol) is a set of networking protocols used for communication over the internet.
- It defines how data is packaged, transmitted, routed, and received between devices.
- TCP ensures reliable and ordered delivery of data, while IP handles addressing and routing.
- It is the standard model used for real-world network communication.



1 Application Layer

The application layer provides network services directly to user applications and interfaces with software like browsers and email clients.
It combines the functions of OSI's application, presentation, and session layers.
It is responsible for protocols like HTTP, HTTPS, FTP, SMTP, and DNS.
This layer handles user requests and prepares data for transmission.

2 Transport Layer

The transport layer ensures end-to-end communication between devices and manages data delivery.
It uses TCP for reliable, ordered communication and UDP for fast, connectionless transmission.
This layer is responsible for port numbers and segmentation of data.
It ensures data reaches the correct application on the destination system.

3 Internet Layer

The internet layer handles logical addressing and routing of packets across networks.
It uses IP addresses to identify source and destination devices.

Protocols like IP, ICMP, and ARP operate at this layer.
It determines the best path for data to travel from source to destination.

4 Network Access Layer

The network access layer manages communication within the local network and physical transmission of data.
It includes both data link and physical layer functions like MAC addressing and signal transmission.
Technologies like Ethernet and Wi-Fi operate at this layer.
It is responsible for sending and receiving data over hardware interfaces.

Why TCP/IP is Used Instead of OSI

TCP/IP was developed for real network communication (ARPANET) and became the standard for the internet.
OSI was designed later mainly as a conceptual framework.

All modern systems, networks, and cloud platforms use TCP/IP for communication.
OSI is mainly used for learning, design understanding, and troubleshooting.

Systemd (Daemon)

Definition

systemd is a system and service manager in Linux that is responsible for booting the system and managing services (daemons).
It controls how services start, stop, and run in the background.

Simple Understanding

```
systemd = boss of all services in Linux
```

- Starts system
- Manages services
- Keeps system running

Why systemd is Used

- Faster boot time
- Centralized service management
- Dependency handling
- Logging (via journal)

Common Commands

Start Service
systemctl start nginx

Stop Service
systemctl stop nginx

Restart Service
systemctl restart nginx

Enable Service (auto start on boot)
systemctl enable nginx

Check Status
systemctl status nginx

Boot Process (High Level): **BIOS → Kernel → systemd → Services**

Logs

Definition

Logs are records of events generated by the operating system, applications, and services.
They help track system activity, errors, and performance for monitoring and troubleshooting.

Simple Understanding

Logs = history of what happened in the system

They answer:

- What happened?
- When?
- Why?

Types of Logs

1 System Logs

Information about OS activities

Location: `/var/log/syslog`

2 Application Logs

Logs generated by applications

Example: `/var/log/nginx/`

3 Security Logs

Authentication and authorization events

Location: `/var/log/auth.log`

4 Kernel Logs

Hardware and kernel messages

Command: `dmesg`

Important Commands

View Logs (Live)

```
tail -f /var/log/syslog
```

View Last Lines

```
tail -n 50 /var/log/syslog
```

Search Logs

```
grep "error" /var/log/syslog
```

View systemd Logs

```
journalctl
```

Service Logs

```
journalctl -u nginx
```

SSH (Secure Shell)

Definition

SSH (Secure Shell) is a protocol used to securely connect to a remote system over a network.
It allows you to access and manage servers using an encrypted connection.

Simple Understanding

```
SSH = remote login to server (securely)
```

Example: You connect your laptop → AWS server

Basic Usage

```
ssh user@server_ip
```

Example:

```
ssh ubuntu@192.168.1.10
```

How SSH Works

1. Client sends request
2. Server responds
3. Authentication happens
4. Secure session established

Authentication Methods

1 Password Authentication

- Enter username + password

2 Key-Based Authentication

- Uses: `Public key + Private key` (More secure)

SSH Key Example

```
ssh-keygen
```

Generates: `id_rsa` (private key) & `id_rsa.pub` (public key)

Connection Example

```
ssh -i key.pem ubuntu@ec2-ip
```

Important SSH Concepts

Port

```
22 (default)
```

Encryption

- Data is encrypted
- Prevents hacking

Daemon: **`sshd`**

Common Commands

Copy file to server

```
scp file.txt user@server:/path
```

Copy from server

```
scp user@server:/file.txt .
```

Check SSH Status

```
systemctl status ssh
```

Common Issues

- Port 22 blocked
- Wrong key
- Permission denied

Shell Scripting (Bash Scripting)

Phase 1 — Linux Command Fundamentals

Before scripting, you must become comfortable with:

- Files
- Directories
- Navigation
- Searching
- Viewing logs
- Permissions

This phase builds your terminal confidence.

1 Navigation Commands

pwd: Shows current directory.

Example Output: /home/ubuntu

ls: Lists files and directories.

ls –l: Detailed listing.

ls –a: Shows hidden files.

cd: Change directory.

Example: cd /var/log

cd .. Go back

2 File & Directory Management

mkdir: Create directory.

Example: mkdir test

touch: Create empty file.

Example: touch file.txt

cp: Copy files.

Example: cp file.txt backup.txt

mv: Move or rename.

Example: mv file.txt newfile.txt

rm: Delete file.

Example: rm file.txt

rm –r: Delete directory recursively.

Example: rm -r test

3 View File Contents

cat: Show full file.

Example: cat file.txt

less: View large file page by page.

Example: less /var/log/syslog

Quit: q

head: First 10 lines.

Example: head file.txt

tail: Last 10 lines.

Example: tail file.txt

tail –f: Live logs monitoring.

Example: tail -f /var/log/syslog

4 Search & Filter Commands

grep: Search text.

Example: grep error app.log

find: Find files/directories.

Example: find /home -name "*.txt"

sort: Sort lines.

Example: sort names.txt

uniq: Remove duplicates.

Example: uniq names.txt

5 Permissions Basics

chmod: Change permissions.

Example: chmod +x script.sh (Makes file executable.)

chown: Change ownership.

Example: sudo chown ubuntu:ubuntu file.txt

6 Compression & Archives

tar: Archive files.

Example: tar -cvf backup.tar mydir

Extract: tar -xvf backup.tar

gzip: Compress file.

Example: gzip file.txt

Extract: gunzip file.txt

7 Monitoring Commands

top: Real-time processes.

Ps: Running processes.

Example: ps aux

df –h: Disk usage.

free –m: Memory usage.

8 Networking Commands

ping: Connectivity check.

Example: ping google.com

curl: Fetch web/API data.

Example: curl ifconfig.me

ip a: Show interfaces.

9 Pipes & Redirection

Pipe |: Sends output of one command to another.

Example: cat file.txt | grep error

Redirect >: To insert data into file

Example: ls > files.txt

Append >>: To add data into file

Example: echo hello >> file.tx

Phase 2 — Shell Scripting Basics Tutorial

Learn how to:

- Write scripts
- Use variables
- Take user input
- Print output
- Run automation tasks (This is where Linux commands become automation.)

1 What is Shell Script?

A shell script is a file containing a sequence of Linux commands executed automatically by the shell.

It is used to automate repetitive tasks, system administration, and application management.

Shell scripts are commonly written using Bash and help simplify command-line operations.

Instead of typing commands one by one: Write once → run anytime

2 Create Your First Script

Step 1: Create file

touch first.sh

Step 2: Open editor

nano first.sh

Step 3: Write script

#!/bin/bash (shebang)

echo "Hello World"

Explanation

#!/bin/bash is called shebang. Tells system **“Run this script using bash shell”**

echo: Prints output.

3 Make Script Executable

chmod +x first.sh

4 Run Script

./first.sh

Output

Hello World

5 Variables

Create Variable

#!/bin/bash

name="Steve"

echo $name

Output

Steve

Rules

No spaces:

name = "Steve" **(wrong)**

Correct: name="Steve"

6 User Input

read

```
#!/bin/bash

echo "Enter your name:"

read name

echo "Hello $name"
```

Output

Enter your name:

Steve

Hello Steve

7 Comments

Single-line comment

This is comment **(**Used for explanation)

Multi-line comments:

```
<<comment
-----------
-----------
Comment              (First and last word should be same)
```

8 Command Substitution

Store command output

```
#!/bin/bash

date_today=$(date)

echo $date_today
```

9 Exit Status

Check command success

```
echo $?
```

Meaning

Value	**Meaning**
0	**Success**
Non-zero	**Error**

10 Special Variables

Variable	Meaning
$0	Script name
$1	First argument
$2	Second argument
$#	Number of arguments
$$	Process ID

Example

Script

```
#!/bin/bash

echo "Script name: $0"

echo "First arg: $1"
```

Run

```
./test.sh hello
```

Output

```
Script name: ./test.sh

First arg: hello
```

11 Simple Real Script

System Information Script

```
#!/bin/bash

echo "Current user:"

whoami

echo "Current directory:"

pwd

echo "System uptime:"

uptime
```

12 Debugging Script

Run with debug

bash -x script.sh (Shows step-by-step execution)

Phase 3 — Conditions & Loops in shell Scripting

Learn how to make scripts:

- Intelligent
- Dynamic
- Decision-based
- Repetitive (automation loops)

This is where scripting becomes real automation.

1 if Condition: “if” is used to make decisions in shell scripts based on conditions.

Basic Syntax

```
if [ condition ]

then
  command
fi
```

Example 1 — Simple Condition

```
#!/bin/bash

num=10

if [ $num -gt 5 ]

then
 echo "Number is greater than 5"
fi
```

Common Operators

Operator	Meaning
-eq	Equal
-ne	Not equal
-gt	Greater than
-lt	Less than
-ge	Greater/equal

-le Less/equal

2 if-else: Executes one block if condition is true, otherwise another block.

Example

```
#!/bin/bash

read -p "Enter number: " num

if [ $num -gt 10 ]

then

  echo "Greater than 10"

else

  echo "Less than or equal to 10"

fi
```

3 if-elif-else: Used for multiple conditions.

Example

```
#!/bin/bash

read -p "Enter marks: " marks

if [ $marks -ge 90 ]

then

  echo "Grade A"

elif [ $marks -ge 70 ]

then

  echo "Grade B"

else

  echo "Grade C"
```

```
fi
```

4 File Conditions

Condition	Meaning
-f	File exists
-d	Directory exists
-r	Readable
-w	Writable
-x	Executable

Example

```
#!/bin/bash
if [ -f test.txt ]
then
  echo "File exists"
else
  echo "File not found"
fi
```

5 for Loop: Used to repeat commands multiple times.

Syntax

```
for variable in list
do
  command
```

```
done
```

Example 1

```
#!/bin/bash

for i in 1 2 3 4 5

do

  echo $i

done
```

Example 2

```
#!/bin/bash

for file in *.txt

do

  echo $file

done
```

6 while Loop: Repeats commands while condition remains true.

Example

```
#!/bin/bash

count=1

while [ $count -le 5 ]

do

  echo $count

  count=$((count+1))
```

```
done
```

7 case Statement: Used as cleaner alternative to multiple if-else conditions.

Example

```
#!/bin/bash

read -p "Enter option: " opt

case $opt in

1)

  date

  ;;

2)

  uptime

  ;;

3)

  df -h

  ;;

*)

  echo "Invalid option"

  ;;

esac
```

8 Logical Operators

Operator	Meaning
&&	AND
`!	NOT

Example

```
if [ $num -gt 5 ] && [ $num -lt 20 ]
then
  echo "Valid"
fi
```

9 Exit Status in Conditions

Example

```
ping -c 1 google.com
if [ $? -eq 0 ]
then
  echo "Reachable"
else
  echo "Unreachable"
fi
```

awk Command

awk is a powerful Linux text-processing tool used to search, filter, extract, and format data from files or command output.

It processes data line by line and splits each line into fields (columns).

Simple Understanding

awk = smart column processor

Best for:

- Logs
- CSV/text files
- Reports
- Structured command output

Basic Syntax

awk 'pattern {action}' file

Example File

Suppose file: employees.txt

contains:

- John 25 DevOps
- Alice 30 AWS
- Bob 28 Linux

Important Concept — Fields

By default:

Space = separator

Field Variables

Variable	Meaning
$1	First column
$2	Second column
$3	Third column
$0	Entire line

1 Print Entire File

```
awk '{print}' employees.txt
```

2 Print Specific Column

Print names

```
awk '{print $1}' employees.txt
```

Output

```
John
Alice
Bob
```

3 Print Multiple Columns

```
awk '{print $1, $3}' employees.txt
```

Output

John DevOps

Alice AWS

Bob Linux

4 Filter Data Using Condition

Print age > 28

```
awk '$2 > 28 {print $1}' employees.txt
```

Output

Alice

5 Pattern Matching

Search Alice

```
awk '/Alice/ {print}' employees.txt
```

6 Using Custom Separator (-F)

Example /etc/passwd

```
awk -F ':' '{print $1}' /etc/passwd
```

: becomes separator.

7 Built-in Variables

Variable	Meaning
NR	Current line number
NF	Number of fields
$0	Full line

Example — Print line numbers

```
awk '{print NR, $1}' employees.txt
```

8 BEGIN and END

BEGIN: Runs before processing starts.

```
awk 'BEGIN {print "Start"}'
```

END: Runs after processing completes.

```
awk 'END {print "Done"}' employees.txt
```

9 Count Lines

```
awk 'END {print NR}' employees.txt
```

10 Sum Numbers

Suppose:

- 10
- 20
- 30

Sum values

```
awk '{sum += $1} END {print sum}' file.txt
```